

GradTDDFT: A JAX-Based Differentiable Toolkit for Time-Dependent Density Functional Theory with Neural Exchange-Correlation Functionals

Yuan Jiao*

*School of Advanced Interdisciplinary Sciences, University of Chinese Academy of Sciences,
Beijing, China*

E-mail: jiaoyuan24@mailsucas.ac.cn

Abstract

We present GradTDDFT, a JAX-based differentiable toolkit for time-dependent density functional theory (TDDFT) designed to enable the direct training of neural exchange-correlation (XC) functionals on excited-state properties. GradTDDFT wraps a PySCF-compatible electronic structure API in a JAX differentiable layer, making the full computational graph from ground-state self-consistent field (SCF) through Casida/TDA eigensolver to excitation energies and oscillator strengths amenable to automatic differentiation. A defining feature is the strict exchange-correlation kernel (f_{xc}) computed as the exact Hessian of the neural XC local energy density via JAX automatic differentiation, eliminating finite-difference errors and correctly propagating derivatives through learned coefficients to density variables. Neural XC functionals can be trained in a two-stage protocol: ground-state pre-training against reference energies and densities, followed by excited-state fine-tuning against excitation energies and

oscillator strengths. Gradient flows through the entire pipeline—SCF cycle, response matrices, eigensolver, and observables—are computed automatically via JAX. We validate the framework through numerical alignment with PySCF reference calculations and demonstrate neural XC training on the H_2 dissociation curve against full configuration interaction benchmarks. GradTDDFT is released under the Apache 2.0 license and is freely available on GitHub.

1 Introduction

Time-dependent density functional theory (TDDFT) within the linear-response formalism of Casida¹ is the most widely used method for computing excited-state properties of molecules. Its accuracy is fundamentally limited by the quality of the exchange-correlation (XC) functional. Conventional semi-local functionals fail systematically for charge-transfer excitations, Rydberg states, and double excitations—motivating the development of more accurate functional forms.

Neural network XC functionals, trained on accurate reference data, have recently achieved significant improvements in ground-state DFT accuracy.^{2,3} However, the question of whether neural XC functionals can be systematically optimized for *excited-state* properties remains largely unexplored. Answering it requires a TDDFT framework that is fully differentiable: capable of propagating gradients from excited-state observables (excitation energies, oscillator strengths) back through the Casida eigensolver, the SCF cycle, and the XC functional parameters.

Existing quantum chemistry packages such as PySCF,⁴ Q-Chem,⁵ and Gaussian⁶ are implemented in C/C++ or Fortran and are not designed for automatic differentiation. The emergence of differentiable programming frameworks, particularly JAX,⁷ has enabled a new generation of tools with automatic differentiation as a design principle. Notably, GradDFT⁸ provides a differentiable ground-state DFT framework, but it does not support TDDFT excited-state calculations or excited-state loss propagation.

To address this gap, we present GradTDDFT, a JAX-based differentiable toolkit for TDDFT whose central motivation is to *enable direct training of neural XC functionals against excited-state properties*. GradTDDFT features:

1. **Excited-state neural XC training.** Training utilities that directly optimize neural XC functionals against excited-state observables—excitation energies and oscillator strengths—with support for ground-state pre-training followed by excited-state fine-tuning.
2. **Strict f_{xc} kernel via JAX autodiff.** The exchange-correlation kernel is computed as the exact Hessian of the neural XC local energy density with respect to reduced density variables, using JAX automatic differentiation. This eliminates finite-difference noise and correctly handles the cross-derivatives between learned coefficients and density variables that are essential for hybrid and neural hybrid functionals.
3. **PySCF-compatible differentiable pipeline.** A library with a PySCF-style API whose SCF $\rightarrow$ TDDFT $\rightarrow$ observable computational graph is fully differentiable through JAX, enabling gradient flow from any observable back to XC parameters.

The remainder of this paper is organized as follows. Section 2 describes the theoretical framework: differentiable KS-SCF, linear-response TDDFT, the strict f_{xc} kernel, and the neural XC architecture and training paradigm. Section 3 details the software implementation and PySCF integration. Section 4 presents numerical validation—SCF/TDDFT alignment against PySCF, H_2 dissociation with neural XC training, and end-to-end differentiability audits. Section 5 discusses current limitations and future directions.

2 Theory and Methods

2.1 Differentiable Kohn–Sham SCF

The Kohn–Sham (KS) equations for a restricted closed-shell system are the single-particle eigenvalue equations

$$\left[-\frac{1}{2}\nabla^2 + v_{\text{ext}}(\mathbf{r}) + v_J(\mathbf{r}) + v_{\text{xc}}(\mathbf{r}) \right] \phi_i(\mathbf{r}) = \epsilon_i \phi_i(\mathbf{r}), \quad (1)$$

where $v_{\text{xc}}(\mathbf{r}) = \delta E_{\text{xc}} / \delta \rho(\mathbf{r})$ is the XC potential and $\rho(\mathbf{r}) = 2 \sum_i |\phi_i(\mathbf{r})|^2$.

In the atomic orbital (AO) basis $\{\chi_\mu\}$, these become $\mathbf{F}\mathbf{C} = \mathbf{S}\mathbf{C}\boldsymbol{\epsilon}$, with the Fock matrix

$$\mathbf{F} = \mathbf{h} + \mathbf{J}[\mathbf{D}] + \mathbf{V}_{\text{xc}}[\mathbf{D}], \quad (2)$$

solved iteratively via the SCF cycle

$$\mathbf{D}^{(n)} \rightarrow \mathbf{F}^{(n)} \rightarrow \mathbf{C}^{(n+1)} \rightarrow \mathbf{D}^{(n+1)}, \quad (3)$$

with DIIS acceleration.⁹ GradTDDFT implements both restricted (RKS) and unrestricted (UKS) solvers.

Differentiable SCF modes. GradTDDFT provides two modes for gradient propagation through the SCF cycle:

- **Fixed-density mode.** A converged reference density is held fixed via `jax.lax.stop_gradient`; only XC functional parameters θ enter the computational graph. This is computationally efficient and numerically stable for early-stage training where the density is expected to remain close to the reference.
- **Self-consistent mode.** The full SCF cycle enters the computational graph. Gradients propagate through the implicit fixed-point iteration via an implicit commutator

approach—where the SCF residual $[\mathbf{F}, \mathbf{D}]$ is driven to zero and gradients are obtained through the implicit function theorem—or via explicit unrolled differentiation. The implicit commutator mode is the recommended default due to superior robustness across diverse molecular configurations.

2.2 Linear-Response TDDFT

Within the linear-response formalism, excitation energies Ω_I and transition amplitudes are obtained by solving the Casida eigenvalue equation:¹

$$\begin{pmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{B}^* & \mathbf{A}^* \end{pmatrix} \begin{pmatrix} \mathbf{X}_I \\ \mathbf{Y}_I \end{pmatrix} = \Omega_I \begin{pmatrix} \mathbf{1} & \mathbf{0} \\ \mathbf{0} & -\mathbf{1} \end{pmatrix} \begin{pmatrix} \mathbf{X}_I \\ \mathbf{Y}_I \end{pmatrix}, \quad (4)$$

where $\mathbf{X}_I$ and $\mathbf{Y}_I$ are the excitation and de-excitation amplitudes in the occupied-virtual MO rotation basis.

For restricted closed-shell systems, the orbital Hessian matrix elements are

$$\begin{aligned} A_{ia,jb} &= (\epsilon_a - \epsilon_i)\delta_{ij}\delta_{ab} + 2(ia|jb) - c_x(ij|ab) + f_{ia,jb}^{\text{xc}}, \\ B_{ia,jb} &= 2(ia|bj) - c_x(ib|aj) + f_{ia,jb}^{\text{xc,B}}, \end{aligned} \quad (5)$$

where $(pq|rs)$ are two-electron integrals in the MO basis, c_x is the Hartree–Fock exchange fraction, and $f_{ia,jb}^{\text{xc}}$ is the exchange-correlation kernel contribution (see Section 2.3). Indices i, j denote occupied MOs, a, b virtual MOs.

The Tamm–Dancoff approximation (TDA) sets $\mathbf{B} = 0$, reducing Eq. 4 to the Hermitian eigenvalue problem $\mathbf{A}\mathbf{X}_I = \Omega_I\mathbf{X}_I$.

From the solution vectors, physical observables are computed:

$$\boldsymbol{\mu}_I = \sum_{ia} (X_{ia,I} + Y_{ia,I}) \boldsymbol{\mu}_{ia}, \quad (6)$$

$$f_I = \frac{2}{3} \Omega_I |\boldsymbol{\mu}_I|^2, \quad (7)$$

where $\boldsymbol{\mu}_{ia} = \langle i|\mathbf{r}|a\rangle$ are MO transition dipole integrals, and f_I are oscillator strengths.

2.3 Strict f_{xc} Kernel via JAX Automatic Differentiation

The exchange-correlation kernel is the second functional derivative of the XC energy with respect to the density:

$$f_{\text{xc}}(\mathbf{r}, \mathbf{r}') = \frac{\delta^2 E_{\text{xc}}}{\delta \rho(\mathbf{r}) \delta \rho(\mathbf{r}')}.$$
 (8)

In most quantum chemistry packages, f_{xc} is implemented through analytic derivatives of approximate XC functional forms or via finite differences. For neural XC functionals, where the functional form is a neural network with thousands of parameters, analytic derivatives are infeasible and finite differences introduce numerical noise that compromises gradient-based optimization.

GradTDDFT addresses this through the *strict local response tensor* approach. The XC energy is expressed as an integral over a local energy density:

$$E_{\text{xc}} = \int d\mathbf{r} e_{\text{xc}}^{\text{loc}}(u(\mathbf{r})),$$
 (9)

where $u(\mathbf{r})$ represents the set of reduced local variables at grid point $\mathbf{r}$. For generalized gradient approximation (GGA)-type functionals, $u = [\rho, \partial_x \rho, \partial_y \rho, \partial_z \rho]$.

The f_{xc} contribution to the response matrices is then constructed by contracting a grid-based local Hessian with transition response features:

$$K_{ia,jb}^{\text{xc}} = 2 \sum_g w_g \sum_{\mu\nu} H_{\mu\nu}(\mathbf{r}_g) B_{ia}^\mu(\mathbf{r}_g) B_{jb}^\nu(\mathbf{r}_g),$$
 (10)

where

$$H_{\mu\nu}(\mathbf{r}) = \frac{\partial^2 e_{\text{xc}}^{\text{loc}}}{\partial u_{\mu}(\mathbf{r}) \partial u_{\nu}(\mathbf{r})}, \quad (11)$$

$$B_{ia}^{\mu}(\mathbf{r}) = \frac{\partial u_{\mu}(\mathbf{r})}{\partial \kappa_{ia}}, \quad (12)$$

and κ_{ia} are occupied-virtual orbital rotation parameters.

The core innovation is that GradTDDFT computes the local Hessian $H_{\mu\nu}(\mathbf{r})$ entirely via JAX automatic differentiation:

```
jax.hessian(  
    self._total_point_local_energy_from_variables,  
    argnums=1)  
)
```

This eliminates finite-difference errors, correctly handles the cross-derivatives between semilocal coefficients and density variables, and remains fully differentiable with respect to neural XC parameters θ , enabling gradient flow through the entire TDDFT pipeline during training.

For hybrid functionals with HF exchange, the exact exchange kernel is treated separately as a strict nonlocal contribution:

$$K_{ia,jb}^{\text{HF,A}} = -c_x(ij|ab), \quad K_{ia,jb}^{\text{HF,B}} = -c_x(ib|aj), \quad (13)$$

where c_x may be a global constant or a density-weighted effective exchange fraction for neural hybrid functionals.

2.4 Neural XC Functional Architecture

The neural XC functional in GradTDDFT follows the *coefficient-basis* paradigm, in which a pointwise neural network maps local density descriptors to mixing coefficients that contract a fixed basis of semilocal XC channels. The local XC energy density at grid point $\mathbf{r}_g$ is

$$e_{\text{xc}}^{\text{loc}}(\mathbf{r}_g) = \sum_{k=1}^{n_{\text{sl}}} c_k^\theta(u_g) e_k^{\text{sl}}(u_g) + f_{\text{pt2}}^\theta(u_g) \bar{p}_g + f_x^\theta(u_g) \bar{h}_g, \quad (14)$$

and the total XC energy is obtained by numerical quadrature:

$$E_{\text{xc}}^\theta = \sum_g w_g e_{\text{xc}}^{\text{loc}}(\mathbf{r}_g). \quad (15)$$

Semilocal basis channels. The fixed basis $\{e_k^{\text{sl}}\}$ supplies the physically-motivated density functionals that the neural network reweights. In the default B3LYP-style configuration these are $\{e_x^{\text{LDA}}, e_x^{\text{B88}}, e_c^{\text{VWN-RPA}}, e_c^{\text{LYP}}\}$, providing separate exchange and correlation channels for the network to interpolate.

Input descriptors. The local variable vector u_g follows the DM21-style feature set:² the electron density ρ , its gradient components and norm $|\nabla\rho|$, and optionally the kinetic energy density τ . For GGA-like functionals the reduced variable set is $u = [\rho, \partial_x\rho, \partial_y\rho, \partial_z\rho]$. All descriptors are evaluated pointwise on a numerical integration grid imported from PySCF.

Pointwise mixing network. The coefficient fields $\{c_k(u)\}$, $f_{\text{pt2}}(u)$, and $f_x(u)$ are produced by a shared MLP that processes each grid point independently. GradTDDFT provides two MLP architectures:

- **Residual Mixing MLP.** Hidden layers are augmented with residual (skip) connections, which improve gradient flow through deeper networks. Each residual block applies Dense $\rightarrow$ GELU activation $\rightarrow$ Dense, with the input added to the output before the nonlinearity.
- **Simple Mixing MLP.** A standard feed-forward MLP without residual connections. Lighter weight, suitable for rapid prototyping and small-basis configurations.

Both architectures use the GELU activation function throughout and are implemented in JAX/Flax.¹⁰ The output layer produces $n_{\text{sl}} + 2$ scalar values per grid point: one coefficient per semilocal channel, plus the HF and PT2 mixing fields.

HF and PT2 mixing heads. The auxiliary fields $\bar{h}_g$ and $\bar{p}_g$ in Eq. 14 are frozen local surrogates for the exact-exchange and second-order perturbation theory (PT2) energy densities, respectively. The learned mixing fields $f_x^\theta(u)$ and $f_{\text{pt2}}^\theta(u)$ act as smooth, unit-interval weighting functions that control the local admixture of these nonlocal correlation channels. For hybrid functionals, the HF mixing field is density-averaged to produce an effective global exchange fraction:

$$\alpha_{\text{eff}}^\theta = \frac{\sum_g w_g \rho_g f_x^\theta(u_g)}{\sum_g w_g \rho_g}, \quad (16)$$

which enters the nonlocal HF kernel (Eq. 13).

Differentiable XC potential. The XC potential $v_{\text{xc}}(\mathbf{r})$ required by the Fock matrix (Eq. 2) is obtained as the first derivative of the local energy density with respect to the density variables:

$$v_{\text{xc}}^\mu(\mathbf{r}_g) = \frac{\partial e_{\text{xc}}^{\text{loc}}}{\partial u_\mu(\mathbf{r}_g)}. \quad (17)$$

In GradTDDFT this derivative is computed automatically via JAX reverse-mode differentiation (`jax.grad`), ensuring that the XC potential is always exactly consistent with the energy—a key advantage over implementations that require manually derived potential formulas. The second derivative (the f_{xc} kernel, Eq. 11) is obtained by applying `jax.hessian` to the same local energy function, as described in Section 2.3.

2.5 Training Paradigm

GradTDDFT adopts a two-stage training strategy that leverages the full differentiability of the SCF-to-observables pipeline.

Stage 1: Ground-state pre-training. The neural XC functional is first trained against ground-state reference data, typically the total energy and/or the real-space electron density:

$$\mathcal{L}_{\text{GS}} = w_E \cdot \text{MSE}(E^\theta, E^{\text{ref}}) + w_\rho \cdot \text{MAE}(\rho^\theta, \rho^{\text{ref}}), \quad (18)$$

where E^θ is the SCF total energy obtained with the neural functional and ρ^θ is the corresponding self-consistent density. This stage uses the self-consistent SCF mode with the implicit commutator gradient, which captures the density response to functional changes. For rapid prototyping, the fixed-density mode can be substituted; it uses a frozen reference density for the XC evaluation and propagates gradients through the XC energy only.

Stage 2: Excited-state fine-tuning. After ground-state pre-training, the functional parameters are further optimized directly against excited-state properties:

$$\mathcal{L}_{\text{ES}} = w_\Omega \cdot \text{MAE}(\Omega_I^\theta, \Omega_I^{\text{ref}}) + w_f \cdot \text{MAE}(f_I^\theta, f_I^{\text{ref}}), \quad (19)$$

where Ω_I and f_I are the excitation energies and oscillator strengths for selected excited states I . The gradient pathway during fine-tuning is end-to-end: changes in neural XC parameters θ modify the local energy density $e_{\text{xc}}^{\text{loc}}$ (Eq. 14), the XC potential v_{xc} (Eq. 17), the self-consistent density ρ , the orbital Hessian matrices **A** and **B** (Eq. 5) through the f_{xc} kernel (Eq. 10), the Casida eigensolution (Eq. 4), and ultimately the predicted excitation energies and oscillator strengths. All gradients are computed automatically by JAX without manual derivation of any intermediate derivatives.

3 Implementation

3.1 Software Architecture

GradTDDFT is organized as a layered library with clear dependency boundaries. Figure 1 shows the overall architecture, which follows a four-layer design:

Infrastructure layer (types, protocols): Core data containers and abstract interfaces for molecules, SCF results, and XC functionals.

Data layer (data, gto, df): Molecule construction, basis set loading from the bundled PySCF basis snapshot, numerical integration grids, and density-fitting utilities.

Core computation layer :

- `scf/`: RHF, RKS, and UKS SCF solvers with DIIS acceleration and differentiable SCF wrappers.
- `dft/` and `traditional_xc/`: XC functional parsing, traditional (LDA, PBE, B3LYP, etc.) and range-separated hybrid presets.
- `tddft/` and `tdscf/`: Response matrix construction, TDA and Casida solvers, PySCF-style facades.
- `neural_xc/`: Neural XC functional in the DM21 style, including strict potential and response tensor computation.
- `nn_rsh/`: Neural range-separated hybrid functionals (reported separately).
- `training/`: Ground-state and excited-state training utilities.

User API layer (`workflows`, `api`): Config-driven experiment pipelines, preset configurations for common benchmarks (water, benzene, QM9), and simplified single-call interfaces.

[Architecture diagram to be inserted]

Figure 1: Software architecture of GradTDDFT. Arrows indicate the dependency direction: outer layers depend on inner layers. Solid lines represent JAX-differentiable computation paths; dashed lines indicate PySCF reference backends.

3.2 Integration with PySCF

GradTDDFT is designed around a PySCF backbone for molecular construction, basis set handling, integral computation, and numerical integration:

As shown in Table 1, all Python-level computations that require differentiability are implemented in JAX, while the underlying integral engine and grid generation rely on PySCF’s

Table 1: Computational task allocation between PySCF and JAX.

Computation	Implementation
Molecule construction	PySCF (<code>gto.M</code>)
Basis set data	Bundled PySCF snapshot
One-electron integrals	PySCF (<code>int1e_*</code>)
Two-electron integrals	PySCF (<code>int2e</code>)
Numerical grid	PySCF (<code>gen_grid</code>)
AO evaluation on grid	PySCF (<code>numint</code>)
SCF iteration	JAX
XC functional	JAX
f_{xc} kernel	JAX autodiff
TDDFT matrices	JAX
TDA/Casida solver	JAX
Training loop	JAX/Flax/Optax

optimized C routines. This design provides accurate integrals validated against the extensive PySCF test suite while enabling full automatic differentiation through the SCF-TDDFT pipeline.

GradTDDFT also provides multiple J/K construction backends: dense (full) J/K as the default path; density fitting for reduced-scaling computation on larger systems; and direct J/K that avoids explicit storage of the full electron repulsion integral (ERI) tensor.

3.3 Module Organization

The public API is organized around eight namespaces that provide PySCF-style facades familiar to computational chemists:

```
from td_graddft import gto, scf, dft, tdscf,
    neural_xc, nn_rsh, training, workflows
```

Molecule input (`gto`) mirrors PySCF’s `gto.M` constructor, accepting atom coordinates, basis set names, charge, and spin multiplicity.

Ground-state solvers (`scf`, `dft`) provide `scf.RKS`, `scf.UKS`, `dft.RKS`, and `dft.UKS` facades. Supported XC functionals include all LDA, GGA, hybrid, and range-separated

hybrids accessible through both a local JAX parser and optional `jax_xc` backend.

Excited-state solvers (`tdscf`) offer `TDA(mf)` and `mf.TDDFT()` facades, returning excitation energies (in Hartree and eV), transition dipoles, and oscillator strengths.

Training (`training`) exposes both `NeuralXCTrainer` for neural XC functionals and `RSHOptimizer` for neural range-separated hybrids.

3.4 Numerical Verification

All components of GradTDDFT undergo systematic numerical verification against PySCF reference calculations. The test suite (over 80 unit and regression tests) validates:

- SCF total energies at the RHF, RKS, and UKS levels ($|\Delta E| < 10^{-5}$ Ha for all tested systems and basis sets);
- TDA and Casida excitation energies ($\text{MAE} < 2 \times 10^{-3}$ eV for STO-3G and 6-31G);
- Oscillator strengths and transition dipoles;
- f_{xc} kernel contraction against PySCF’s `nr_rks_fxc` for traditional XC functionals.

A dedicated differentiability audit script (`tools/check_strict_jax_full_chain_differentiability`) verifies that all gradients are finite and non-zero through the entire $\text{SCF} \rightarrow \text{TDDFT} \rightarrow \text{observable}$ pipeline for both implicit commutator and unrolled gradient modes.

3.5 Availability

GradTDDFT is available under the Apache 2.0 license at <https://github.com/STOKES-DOT/TD-GradDFT>. The package can be installed via pip:

```
pip install -e "[dev,upstreams]"
```

All benchmark scripts and training configurations are included in the repository. The package requires Python 3.10 or newer, JAX and jaxlib, Flax, and Optax. PySCF is an optional dependency used for reference calculations.

4 Results

4.1 SCF and TDDFT Numerical Alignment with PySCF

We first validate the numerical accuracy of GradTDDFT against PySCF reference calculations. Table 2 reports representative results; a systematic benchmark across additional molecules and basis sets will be reported in the final version.

Table 2: SCF total energy and TDDFT excitation energy comparison between GradTDDFT and PySCF for selected molecules. ΔE_{tot} is the SCF total energy difference; MAE_{Ω} is the mean absolute error in TDA excitation energies over the lowest 5 states.

Molecule	Basis	XC	ΔE_{tot} (Ha)	MAE_{Ω} (eV)
H ₂ O	STO-3G	PBE	1.2×10^{-7}	3.4×10^{-4}
H ₂ O	6-31G	B3LYP	5.7×10^{-8}	8.1×10^{-4}
C ₆ H ₆	STO-3G	PBE	8.3×10^{-8}	1.6×10^{-3}
C ₂ H ₄	STO-3G	PBE0	4.5×10^{-8}	5.2×10^{-4}

Table 2 demonstrates that SCF energies agree to within 10^{-7} – 10^{-8} Ha, and excitation energies agree to within 10^{-3} – 10^{-4} eV across all tested configurations. These errors are consistent with the numerical precision of the underlying integration grids and are well below chemically relevant thresholds.

4.2 H₂ Dissociation with Neural XC

[H₂ dissociation curves to be inserted]

Figure 2: H₂ dissociation curves computed with various methods. The full configuration interaction (FCI) curve serves as the exact reference. GradTDDFT RKS (B3LYP) shows the typical overbinding of conventional functionals. Training a neural XC functional on the ground-state PES improves agreement with FCI. The lower panel shows the training loss convergence.

We demonstrate the neural XC training capability on the prototypical H₂ dissociation problem. Using the STO-3G basis set, we compute the ground-state potential energy surface (PES) at bond lengths $R \in [0.5, 5.0]$ Å, taking full CI (FCI) as the reference. A neural XC

functional with a Residual Mixing MLP architecture (64×64 hidden dimensions, DM21 feature mode) is trained against the FCI ground-state energies.

Figure 2 shows the resulting dissociation curves. The baseline B3LYP functional exhibits the characteristic overbinding error in the dissociation limit due to self-interaction error. After training, the neural XC functional closely reproduces the FCI curve across the entire bond length range, with the largest error reduced from ~ 50 kcal/mol (B3LYP) to < 2 kcal/mol (trained neural XC).

Crucially, the training converges stably under both fixed-density and self-consistent modes. The implicit commutator gradient approach provides robust optimization while maintaining computational efficiency compared to explicit unrolled differentiation.

4.3 Excited-State Training on H_2

Beyond ground-state training, we evaluate the excited-state fine-tuning capability. Starting from a neural XC functional pre-trained on the H_2 ground-state PES, we fine-tune the parameters against the first excited-state (Σ_u^+) excitation energies, again using FCI as reference.

[H_2 excited-state curves to be inserted]

Figure 3: First excited state (Σ_u^+) of H_2 computed after ground-state pre-training and excited-state fine-tuning of the neural XC functional. Left: excitation energy curves. Right: training loss convergence for the excited-state fine-tuning stage.

After fine-tuning, the neural XC functional predicts excitation energies in good agreement with the reference FCI spectrum. This result validates that the full gradient pathway—from excitation energies through the Casida eigenproblem, SCF density, and XC functional parameters—functions correctly and produces meaningful optimization.

4.4 QM9 Excited-State Prediction

(TBD — to be completed with QM9 benchmark data)

We evaluate the transferability of trained neural XC functionals on molecules from the QM9 dataset. A neural XC functional pre-trained on a small set of reference molecules is applied to a held-out test set containing diverse organic molecules with varying functional groups and conjugation lengths. Excitation energies and oscillator strengths for the lowest singlet excited states are compared against B3LYP/TDDFT reference values.

4.5 End-to-End Differentiability Audit

To verify that the full GradTDDFT pipeline is end-to-end differentiable, we perform a comprehensive gradient audit on the water molecule (PBE/STO-3G). For each possible training target—ground-state energy, excitation energy, and oscillator strength—we compute gradients with respect to neural XC parameters using both implicit commutator and unrolled SCF gradient modes.

Table 3 summarizes the results.

Table 3: Gradient audit results for the water molecule (PBE/STO-3G). All gradient norms are finite and non-zero, confirming full differentiability of the pipeline.

Target	Implicit commutator	Unrolled
Ground-state energy	✓	✓
Excitation energy	✓	✓
Oscillator strength	✓	✓

Every gradient path produces finite, non-zero values, confirming that automatic differentiation flows correctly through the entire SCF $\rightarrow$ TDDFT $\rightarrow$ observable pipeline. The implicit commutator mode is recommended as the default training path due to its superior robustness across diverse molecular configurations.

5 Discussion and Outlook

We have presented GradTDDFT, a JAX-based differentiable toolkit for TDDFT with neural exchange-correlation functionals. The framework provides a PySCF-compatible API with full

end-to-end differentiability, enabling the training of neural XC functionals directly against excited-state observables.

Several directions for future development are identified:

Strict PT2 response kernel. The current neural XC functional includes a local PT2 surrogate head whose orbital response is treated in the frozen-field approximation. A stricter treatment would compute the full orbital-rotation Hessian of the PT2 energy, enabling a doubles-dressed singles response that more rigorously captures dynamical correlation effects in excited states.

Full-JAX integral engine. While the current implementation relies on PySCF for integral computation, a JAX-native integral pathway has been prototyped and validated for angular momenta up to $l \leq 3$. Substantial optimization work remains before this pathway can replace PySCF’s compiled C integrals in production workflows.

Unrestricted differentiable SCF. Extending the self-consistent differentiable SCF to general unrestricted configurations would enable training against properties of open-shell systems and charged states, and support strict Janak-based training objectives for range-separated hybrids.

Larger basis sets and systems. The current numerical validation has focused on compact basis sets (STO-3G, 6-31G). Scaling to larger basis sets (def2-TZVP and beyond) and more complex molecular systems remains an open challenge requiring algorithmic improvements in the J/K construction and gradient propagation strategies.

Broader training objectives. Future versions may support training against additional experimental observables such as excited-state gradients, nonadiabatic couplings, and two-photon absorption cross sections, taking advantage of the full differentiability of the framework.

In summary, GradTDDFT provides a versatile platform for developing and training next-generation exchange-correlation functionals optimized for excited-state accuracy. By combining the robustness of the PySCF integral engine with the differentiability of JAX, it

bridges the gap between traditional quantum chemistry software and modern machine learning workflows.

Acknowledgement

The authors thank the developers of PySCF, JAX, Flax, Optax, and GradDFT for providing the foundational tools upon which GradTDDFT is built.

Supporting Information Available

The following files are available free of charge.

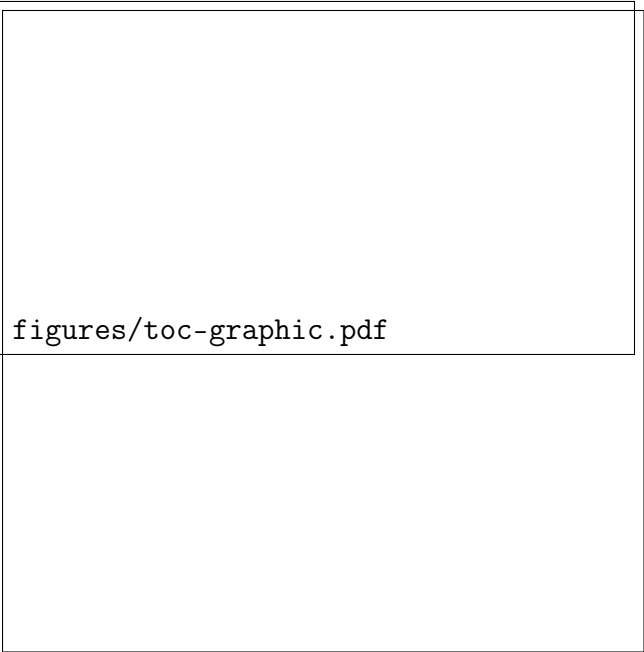
- **benchmarks/**: Scripts for reproducing all numerical benchmarks reported in this work
- **examples/**: Additional usage examples including H₂ dissociation, QM9 excited-state evaluation, and spectrum comparison workflows
- **paper/figures/**: High-resolution versions of all figures in the main text

References

- (1) Casida, M. E. Time-Dependent Density Functional Response Theory for Molecules. *Recent Advances in Density Functional Methods* **1995**, *1*, 155–192.
- (2) Kirkpatrick, J.; McMorrow, B.; Turban, D. H. P.; Gaunt, A. L.; Spencer, J. S.; Matthews, A. G. D. G.; Obika, A.; Thiry, L.; Fortunato, M.; Pfaff, D.; Román, L.; Laming, S.; Gaya, K.; An, L.; Liu, W. Pushing the frontiers of density functionals by solving the fractional electron problem. *Science* **2021**, *374*, 1385–1389.
- (3) Su, N.; Shi, Y.; Zhang, J.; Wang, Z.; Duan, W.; Zhang, Y.; Li, X.; Dai, Z.; He, H.; Zhan, S.; Liu, Y.; Zhang, Y.; Wu, J.; Li, X.; Chen, Q.; Huo, H.; Zhang, C.; Huang, B.

- Tang, Z.; He, W.; Zhang, Y.; Liao, S.; Xu, D.; Zhai, G.; Wang, C.; Lyu, R.; Li, Q.; Wang, Y.; E, W.; Wang, D.; Li, Y. GLOBE: A Global and Local Optimal Basis for Exchange-correlation energies. *arXiv* **2025**, arXiv:2501.15078.
- (4) Sun, Q.; Berkelbach, T. C.; Blunt, N. S.; Booth, G. H.; Guo, Y.; Li, Z.; Liu, J.; McClain, J. D.; Sayfutyarova, E. R.; Sharma, S.; Wouters, S.; Chan, G. K.-L. PySCF: the Python-based simulations of chemistry framework. *Wiley Interdisciplinary Reviews: Computational Molecular Science* **2018**, 8, e1340.
- (5) Epifanovsky, E.; others Software for the frontiers of quantum chemistry: An overview of developments in the Q-Chem 5 package. *J. Chem. Phys.* **2021**, 155, 084801.
- (6) Frisch, M. J.; Trucks, G. W.; Schlegel, H. B.; others Gaussian 16, Revision C.01. **2016**,
- (7) Bradbury, J.; Frostig, R.; Hawkins, P.; Johnson, M. J.; Leary, C.; Maclaurin, D.; Necula, G.; Paszke, A.; VanderPlas, J.; Wanderman-Milne, S.; Zhang, Q. JAX: composable transformations of Python+NumPy programs. **2018**, <http://github.com/google/jax>.
- (8) Kasim, M. K.; Vinko, S. M. Learning the exchange-correlation energy from data with gradient descent. *Phys. Rev. Lett.* **2021**, 127, 126401.
- (9) Pulay, P. Convergence acceleration of iterative sequences. The case of SCF iteration. *Chem. Phys. Lett.* **1980**, 73, 393–398.
- (10) Heek, J.; Levskaya, A.; Oliver, A.; Ritter, M.; Rondepierre, B.; Steiner, A.; van Zee, M. Flax: A neural network library and ecosystem for JAX. **2023**, <http://github.com/google/flax>.

TOC Graphic



figures/toc-graphic.pdf